

Report HW 6 ECE-111

Name: Jasper Huang

PID: A17796149

Homework 6a (Vending Machine)

Mealy FSM

1. SystemVerilog design code

```
always_ff@(posedge clk) begin
    if(!rstn) begin
        // Student to Add Code
        present_state <= CENTS_0;
        r_N <= 0;
        r_D <= 0;
    end
    else begin
        // Student to Add Code
        present_state <= next_state;
        r_N <= N;
        r_D <= D;
    end
end
end
```

```
always_comb begin
    // Student to Add Code
    next_state = present_state;
    open = 0;
    case (present_state)
        CENTS_0: begin
            if (r_N) next_state = CENTS_5;
            else if (r_D) next_state = CENTS_10;
        end

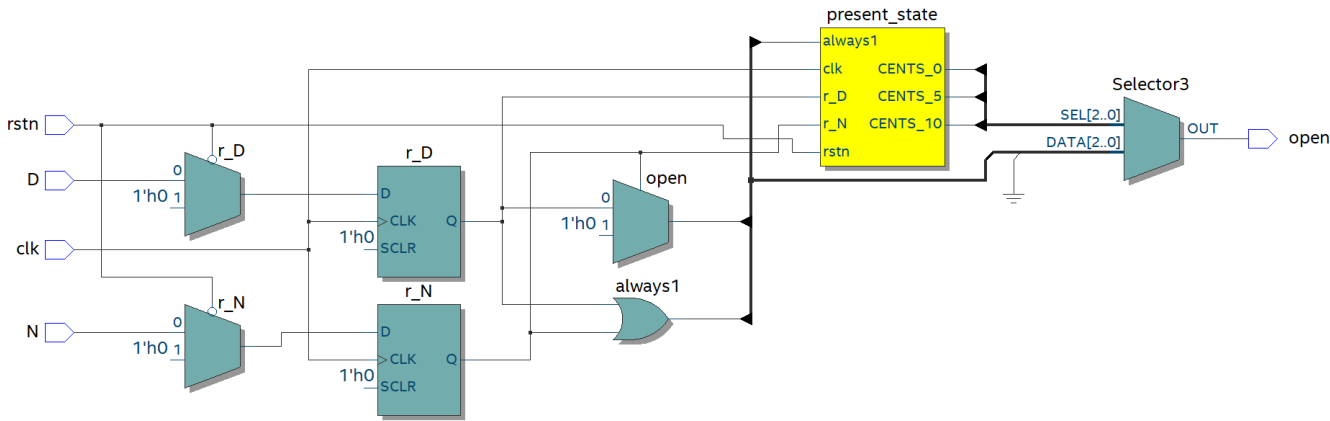
        CENTS_5: begin
            if (r_N) next_state = CENTS_10;
            else if (r_D) begin
                next_state = CENTS_0;
                open = 1;
            end
        end

        CENTS_10: begin
            if (r_N || r_D) begin
                next_state = CENTS_0;
                open = 1;
            end
        end

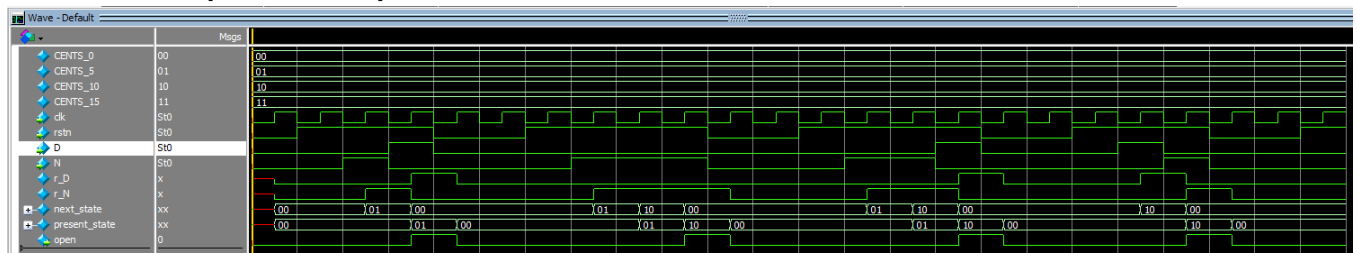
        default: begin
            next_state = present_state;
            open = 0;
        end
    endcase
end
```

2. Synthesis resource usage and schematic generated from RTL netlist viewer

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	4
2		
3	▼ Combinational ALUT usage for logic	6
1	-- 7 input functions	0
2	-- 6 input functions	2
3	-- 5 input functions	1
4	-- 4 input functions	1
5	-- <=3 input functions	2
4		
5	Dedicated logic registers	5
6		
7	I/O pins	5
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	rstn~input
12	Maximum fan-out	5
13	Total fan-out	41
14	Average fan-out	1.95



3. Simulation snapshot and explain simulation result



The simulation wave form shows after inserting Nickle or Dime (High N or High D), the state changes and as it reaches a total of 15, the open flag is high. The implementation remove the state CENTS_15 as it's unnecessary for the vending machine. The open flag is set correctly for the input D and N.

Moore FSM

1. SystemVerilog design code

```
// state variables and state encoding parameters
parameter[3:0] CENTS_0=4'b0001, CENTS_5=4'b0010, CENTS_10=4'b0100, CENTS_15=4'b1000;
logic[3:0] present_state, next_state;

// Sequential Logic for present state
always_ff@(posedge clk) begin
    // Student to Add Code
    if(!rstn) begin
        present_state <= CENTS_0;
    end else begin
        present_state <= next_state;
    end
end

// Combination Logic for Next State and Output
always_comb begin
    // Student to Add Code
    next_state = present_state;
    open = 0;

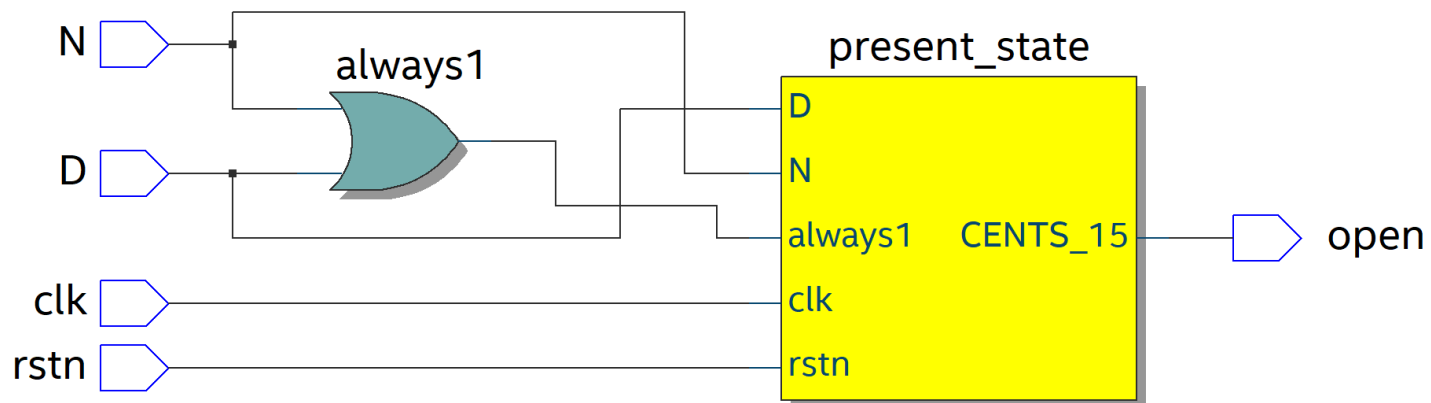
    case (present_state)
        CENTS_0: begin
            if (N) next_state = CENTS_5;
            else if (D) next_state = CENTS_10;
        end
        CENTS_5: begin
            if (N) next_state = CENTS_10;
            else if (D) next_state = CENTS_15;
        end
        CENTS_10: begin
            if (N || D) next_state = CENTS_15;
        end
        CENTS_15: begin
            open = 1;
            if (rstn == 0) begin
                next_state = CENTS_0;
            end else begin
                next_state = CENTS_15;
            end
        end
        default: begin
            next_state = CENTS_0;
        end
    endcase
end
```

2. Synthesis resource usage and schematic generated from RTL netlist viewer

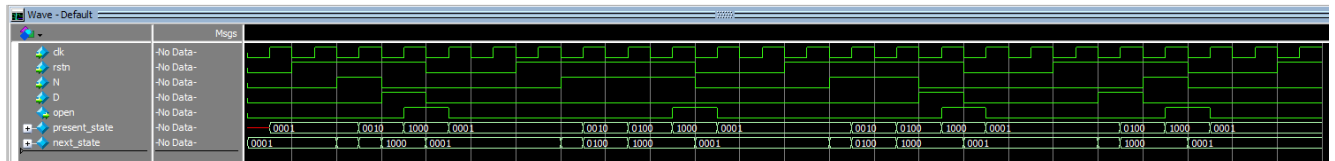
Analysis & Synthesis Resource Usage Summary

<<Filter>>

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	3
2		
3	Combinational ALUT usage for logic	4
1	-- 7 input functions	0
2	-- 6 input functions	2
3	-- 5 input functions	1
4	-- 4 input functions	1
5	-- <=3 input functions	0
4		
5	Dedicated logic registers	4
6		
7	I/O pins	5
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	rstn~input
12	Maximum fan-out	4
13	Total fan-out	35
14	Average fan-out	1.94

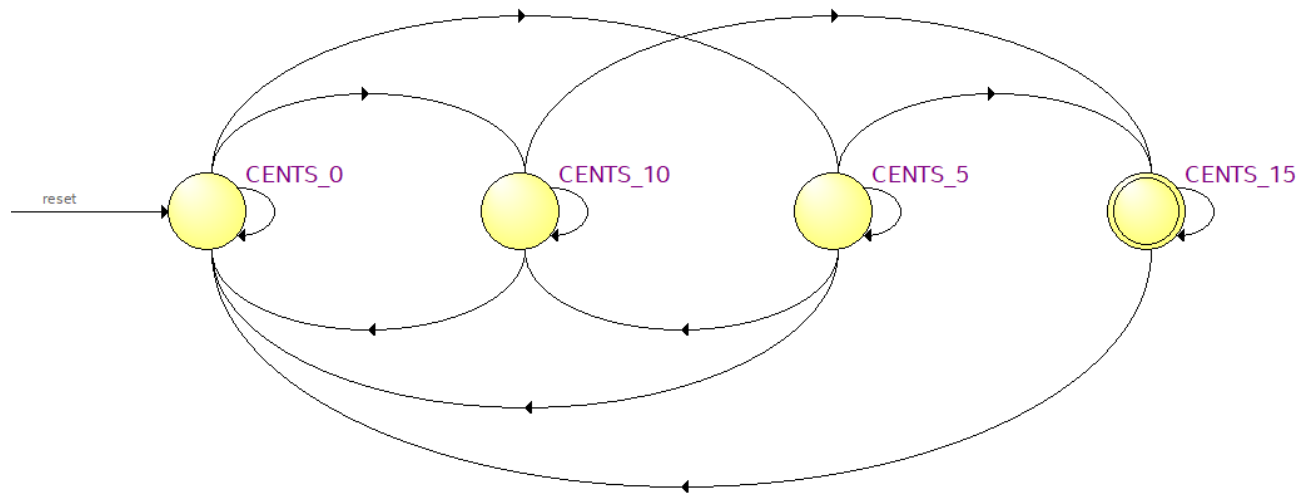


3. Simulation snapshot and explain simulation result



The simulation wave form shows after inserting Nickle or Dime (High N or High D), the state changes and as it reaches a total of 15, the open flag is high. The open flag is set correctly for the input D and N. The present state also correctly reflects on the previous next state.

4. FSM state transition diagram



	Source State	Destination State	Condition
1	CENTS_0	CENTS_0	(!rstn) + (rstn).(!D).(!N)
2	CENTS_0	CENTS_10	(D).(!N).(rstn)
3	CENTS_0	CENTS_5	(N).(rstn)
4	CENTS_5	CENTS_15	(rstn).(D).(!N)
5	CENTS_5	CENTS_0	(!rstn)
6	CENTS_5	CENTS_10	(N).(rstn)
7	CENTS_5	CENTS_5	(!D).(!N).(rstn)
8	CENTS_10	CENTS_15	(rstn).(always1)
9	CENTS_10	CENTS_0	(!rstn)
10	CENTS_10	CENTS_10	(!always1).(rstn)
11	CENTS_15	CENTS_15	(rstn)
12	CENTS_15	CENTS_0	(!rstn)

Homework 6b (Integer Multiplier)

SystemVerilog FSM design code (Student Added Parts)

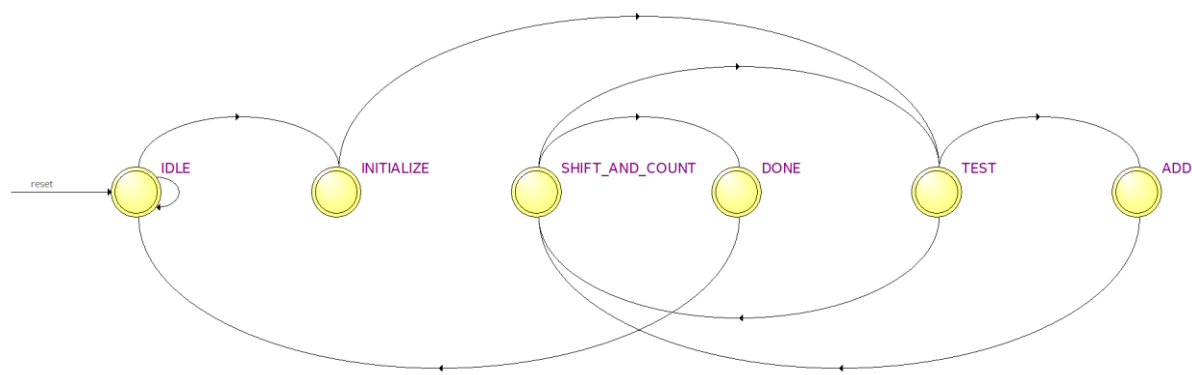
```
// Instantiate N-bit carry lookahead adder
// Pass add_operand1, add_operand2 and sum
// Tie CIN to '0'
carry_lookahead_adder #(N(N)) adder_inst(
    // Student to add code here
    // use add_operand1, add_operand2, sum logic (wires) to connect to carry lookahead adder inputs
    .A(add_operand1),
    .B(add_operand2),
    .CIN(0),
    .result(sum)
);
```

```
IDLE: begin
    // Student to add code here
    count <= 0;
    load_reg <= 0;
    shift_reg <= 0;
    if (start) begin
        next_state <= INITIALIZE;
    end else begin
        next_state <= IDLE;
    end
end
// Load Multiplicand and Multiplier in a load register and a shift register
// Initialize count to 0 and then set next_state to TEST
INITIALIZE: begin
    shift_reg <= {1'b0, {N{1'b0}}}, multiplier};
    // Student to add remaining code in INITIALIZE state
    load_reg <= multiplicand;
    next_state <= TEST;
    count <= 0;
end
```

```
TEST: begin
    if(shift_reg[0] == 1'b1) begin
        // Pass Multiplicand to carry lookahead adder one of the input (add_operand1)
        // Pass to carry lookahead adder second input add_operand2, the previous stage adder output value to add with Multiplicand. (shift_reg[(2*N)-1:N])
        // Move to ADD state
        // Student to add code here
        add_operand1 <= multiplicand;
        add_operand2 <= shift_reg[(2*N)-1:N];
        next_state <= ADD;
    end
    else begin
        // Since no add operation to be performed pass 0 to carry lookahead input one of the input (add_operand1)
        // Pass to carry lookahead adder second input add_operand1, the previous stage adder output value (shift_reg[(2*N)-1:N])
        // move to shift and increment count state (SHIFT_AND_COUNT)
        // Student to add code here
        add_operand1 <= 0;
        add_operand2 <= shift_reg[(2*N)-1:N];
        next_state <= SHIFT_AND_COUNT;
    end
end
ADD: begin
    shift_reg <= {sum, shift_reg[N-1:0]}; // Load shift register : Output sum from Adder which includes carry and retain previous lower bit of shift register
    // move to shift and increment count state (SHIFT_AND_COUNT)
    // Student to add code here
    next_state <= SHIFT_AND_COUNT;
end
```

```
SHIFT_AND_COUNT: begin
    // Right shift entire shift_reg by 1 position and store result in shift_reg
    // Increment count
    // student to add code here
    shift_reg <= shift_reg >> 1;
    count <= count + 1;
    if(count == N-1) begin // If 'N' times SHIFT operation performed then move to Done state else go back to Test state
        // Student to add code here
        next_state <= DONE;
    end
    else begin
        // Student to add code here
        next_state <= TEST;
    end
end
```

State Transition Diagram



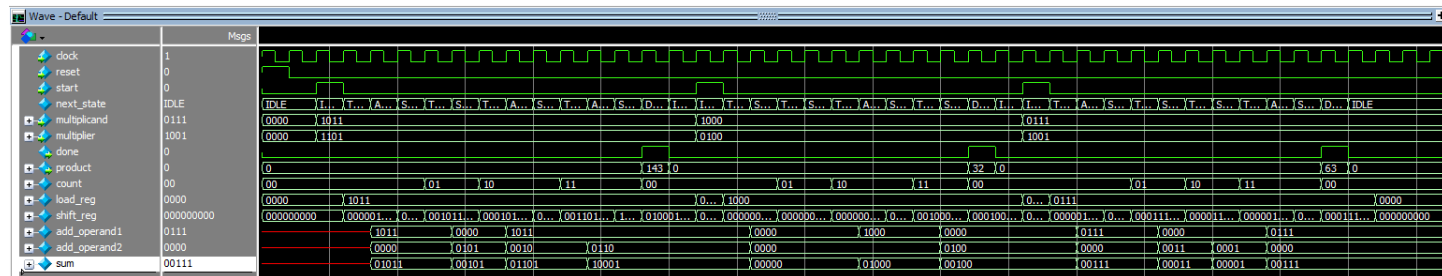
State Transition Table

	Source State	Destination State	Condition
1	ADD	SHIFT_AND_COUNT	
2	DONE	IDLE	
3	IDLE	IDLE	(!start)
4	IDLE	INITIALIZE	(start)
5	INITIALIZE	TEST	
6	SHIFT_AND_COUNT	TEST	(!count[0]) + (count[0]).(!count[1])
7	SHIFT_AND_COUNT	DONE	(count[0]).(count[1])
8	TEST	ADD	(shift_reg[0])
9	TEST	SHIFT_AND_COUNT	(!shift_reg[0])

Synthesis resource usage

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	21
2		
3	Combinational ALUT usage for logic	36
1	-- 7 input functions	0
2	-- 6 input functions	5
3	-- 5 input functions	2
4	-- 4 input functions	8
5	-- <=3 input functions	21
4		
5	Dedicated logic registers	25
6		
7	I/O pins	21
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clock-input
12	Maximum fan-out	25
13	Total fan-out	232
14	Average fan-out	2.25

Simulation snapshot & Transcript & Explanation



The simulation result shows the transition between 6 states. The load_reg and shift_reg also changes correctly based on the correct operation determined in the TEST stage and the output from the carry_lookahead_adder. The state changes in between TEST ADD SHIFT_AND_COUNT according to the TEST result and continue until the product is calculated and the done flag is up once the cycle finished. The entire simulation produce three products which are correct based on the input multiplicand and multiplier.

Homework 6c (Integer Multiplier)

SystemVerilog FSM design code (Student Added Parts)

```
// Instantiate (N)-bit carry lookahead adder
carry_lookahead_adder #(.N(N)) adder_inst(
    // Student to add code
    .A(add_operand1),
    .B(add_operand2),
    .CIN(carry),
    .result(sum)
);
```

```
case(next_state)
// Wait for start signal
IDLE: begin
    // Student to add code
    if (start) begin
        next_state <= INITIALIZE;
    end else begin
        next_state <= IDLE;
    end
    shift_reg <= 0;
end

// Load Multiplicand and Multiplier in a load register and a shift register
INITIALIZE: begin
    // load multiplicand to load_reg_pos
    // Student to Add Code Here
    load_reg_pos <= multiplicand;

    // load multiplicand_neg[N:0] to load_reg_neg
    // Student to Add Code Here
    load_reg_neg <= multiplicand_neg[N-1:0];

    shift_reg <= {(N{1'b0}), multiplier, 1'b0};
    next_state <= TEST;
    count <= 0;
end
```

```
TEST: begin
    if(shift_reg[1:0] == 2'b01) begin
        // Pass positive Multiplicand to carry lookahead adder input
        // Pass previous adder output value after shift to add with Multiplicand
        // move to add state
        // Student to add code here
        add_operand1 <= multiplicand;
        add_operand2 <= shift_reg[(2*N):(N+1)];
        next_state <= ADD;
    end
    else if(shift_reg[1:0] == 2'b10) begin
        // Pass negative Multiplicand to carry lookahead adder input
        // Pass previous adder output value after shift to add with Multiplicand
        // move to add state

        // Student to Add code here
        add_operand1 <= multiplicand_neg;
        add_operand2 <= shift_reg[(2*N):(N+1)];
        next_state <= ADD;
    end
    else begin
        // assign add_operand1 to 0, Since no add operation to be perform pass 0 to carry lookadder input
        // Pass previous adder output value after shift to add with Multiplicand
        // move to shift and increment count state

        // Student to Add code
        add_operand1 <= 0;
        add_operand2 <= shift_reg[(2*N):(N+1)];
        next_state <= SHIFT_AND_COUNT;
    end
end
```

```
ADD: begin
    shift_reg <= {sum, shift_reg[N:0]}; // Load shift register : Output sum from Adder which includes carry
    // Move to shift and increment count state
    // Student to add code here
    next_state <= SHIFT_AND_COUNT;
end

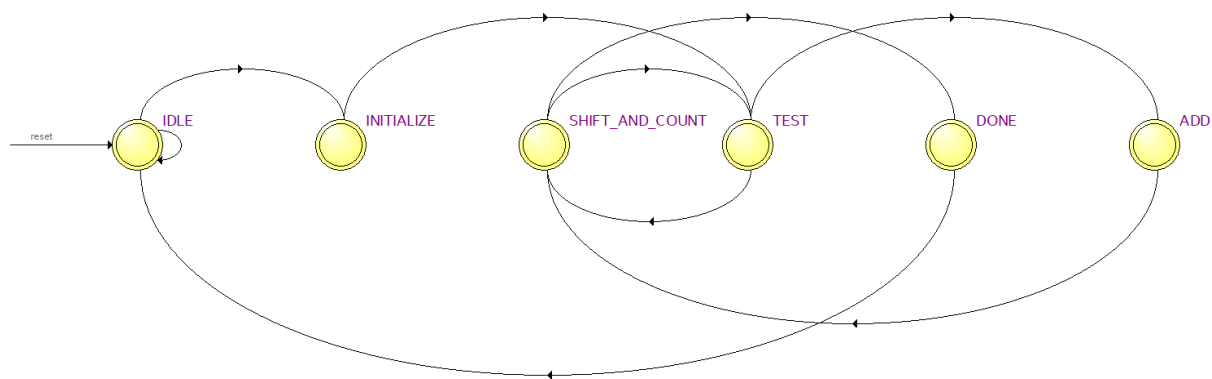
SHIFT_AND_COUNT: begin
    shift_reg <= (shift_reg >> 1); // Right Arithmetic shift entire shift register by 1 position

    // Increment count
    // Student to Add code here
    count <= count + 1;

    if (count == N-1) begin // If 'N' times SHIFT operation performed then move to Done state else go back
        // Student to Add code here
        next_state <= DONE;
    end
    else begin
        // Student to Add code here
        next_state <= TEST;
    end
end

DONE: begin
    next_state <= IDLE; // Wait for right shift value to be available. This is the final product value.
end
endcase
```

State Transition Diagram

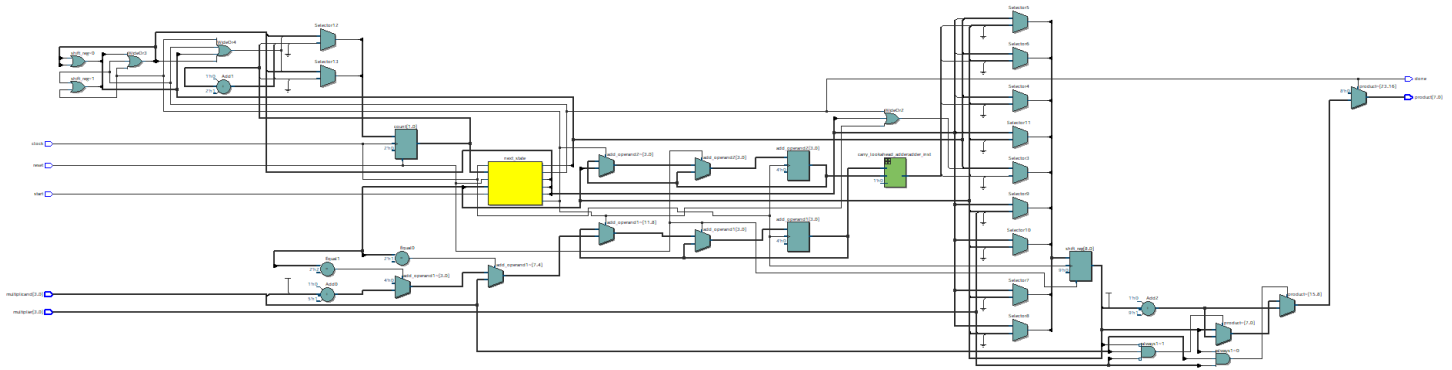


	Source State	Destination State	Condition
1	ADD	SHIFT_AND_COUNT	
2	DONE	IDLE	
3	IDLE	IDLE	(!start)
4	IDLE	INITIALIZE	(start)
5	INITIALIZE	TEST	
6	SHIFT_AND_COUNT	DONE	(count[0]).(count[1])
7	SHIFT_AND_COUNT	TEST	(!count[0]) + (count[0]). (!count[1])
8	TEST	ADD	(!shift_reg[0]). (shift_reg[1]) + (shift_reg[0]). (!shift_reg[1])
9	TEST	SHIFT_AND_COUNT	(!shift_reg[0]). (!shift_reg[1]) + (shift_reg[0]). (shift_reg[1])

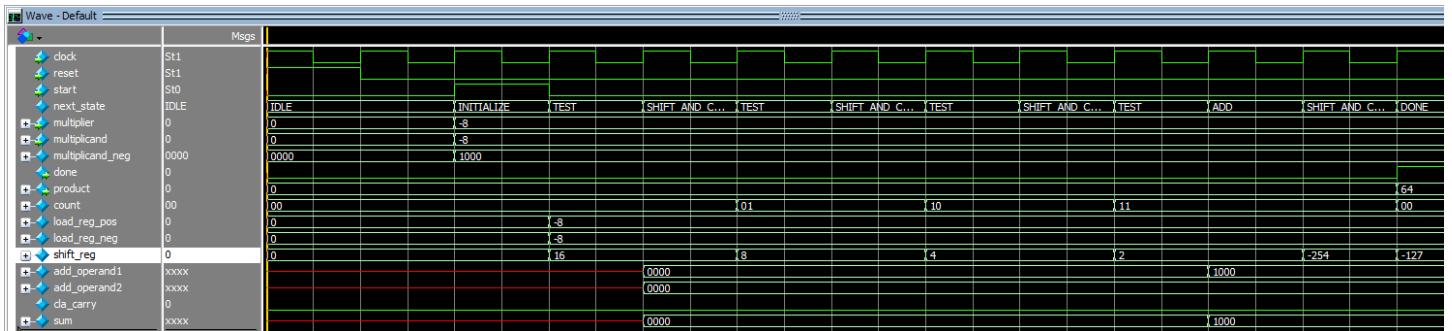
Synthesis Resource Usage

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	25
2		
3	Combinational ALUT usage for logic	44
1	-- 7 input functions	0
2	-- 6 input functions	4
3	-- 5 input functions	2
4	-- 4 input functions	19
5	-- <=3 input functions	19
4		
5	Dedicated logic registers	25
6		
7	I/O pins	20
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clock~input
12	Maximum fan-out	25
13	Total fan-out	273
14	Average fan-out	2.50

RTL netlist viewer



Simulation snapshot & Explanation & Transcript



```
#
# time=64250000 Multiplicand= 7 Multiplier= -2 Product= -14 Done=1 : Correct Result
#
# time=64490000 Multiplicand= 7 Multiplier= -1 Product= -7 Done=1 : Correct Result
#
# time=64710000 Multiplicand= 7 Multiplier= 0 Product= 0 Done=1 : Correct Result
#
# time=64970000 Multiplicand= 7 Multiplier= 1 Product= 7 Done=1 : Correct Result
#
# time=65230000 Multiplicand= 7 Multiplier= 2 Product= 14 Done=1 : Correct Result
#
# time=65490000 Multiplicand= 7 Multiplier= 3 Product= 21 Done=1 : Correct Result
#
# time=65750000 Multiplicand= 7 Multiplier= 4 Product= 28 Done=1 : Correct Result
#
# time=66050000 Multiplicand= 7 Multiplier= 5 Product= 35 Done=1 : Correct Result
#
# time=66310000 Multiplicand= 7 Multiplier= 6 Product= 42 Done=1 : Correct Result
#
# time=66570000 Multiplicand= 7 Multiplier= 7 Product= 49 Done=1 : Correct Result
#
# Error Count = 0, Test Passed !
#
# ** Note: $finish : C:/Users/Jaspe/repositories/ECE 111 Lab HW/Homework6/Lab6/booth_multiplier/booth_multiplier_testbench.sv(102)
# Time: 66800 ns Iteration: 0 Instance: /booth_multiplier_testbench
```

The simulation result shows all tests passed from the transcript. The above simulation wave is the first test case where you can see the next_state changes correctly between each determined test shift_reg[1:0] and update the shift_reg accordingly until the counter reaches the N count and return the product. The wave shows -8 * -8 which gives 64 in the product, the model works perfectly as a integer multiplier.